USE PYTORCH TO BUILD NEURAL NETWORKS

Key Concepts in PyTorch

- **PyTorch** is an open-source deep learning framework.
- Allow flexible, dynamic computation graph, and ease of use.
- PyTorch is widely used in both academia and industry for research and production.
- Integration with Python: PyTorch operates more like a Python library than a framework.

- **Tensors:** Multi-dimensional arrays that are the fundamental building blocks in PyTorch.
- **Autograd:** Automatic differentiation system for computing gradients (that powers neural network training).
- **NN Module:** The core building block for creating neural networks.
- **Optimizers:** Algorithms for adjusting the parameters of the network to minimize loss.

How to Install PyTorch in Anaconda with Conda or Pip

1. Open the Anaconda Prompt or Terminal.
2. Create a new conda environment for PyTorch:

conda create --name pytorch_env

3. Activate the new environment:

conda activate pytorch_env

4. Install PyTorch:

conda install pytorch torchvision torchaudio -c pytorch

pip install torch torchvision torchaudio

5. Open Python IDE to Verify the installation:

import torch

print(torch.__version__)

```
import torch
print(torch.__version__)
```

```
2.4.0
```

Example: Building a Simple Neural Network

Define a simple neural network with PyTorch's **nn.Module**.

Example: A fully connected network for a classification task.

e.g.:
self.fc1 = nn.Linear(784, 128) # 784 input features to 128 neurons

```python
@author: mireilla
"""

import torch   #the core PyTorch library
import torch.nn as nn   ##contains the building blocks for neural networks.
import torch.nn.functional as F   ##provides functions for different operations, including activation functions.

# Define the neural network class.
# The network is defined by creating a subclass of nn.Module,
# which is the base class for all neural network modules in PyTorch.
class SimpleNN(nn.Module):
    # The __init__ method defines the layers of the network
    def __init__(self):
        super(SimpleNN, self).__init__()

        # Define the layers
        self.fc1 = nn.Linear(784, 128)  # First fully connected layer (input: 784, output: 128)
        self.fc2 = nn.Linear(128, 64)   # Second fully connected layer (input: 128, output: 64)
        self.fc3 = nn.Linear(64, 10)    # Third fully connected layer (input: 64, output: 10)

    #  The forward method defines how the input data flows through the network.
    def forward(self, x):
        # Apply ReLU activation to the first layer
        x = F.relu(self.fc1(x))

        # Apply ReLU activation to the second layer
        x = F.relu(self.fc2(x))

        # Apply softmax activation to the output layer
        x = F.log_softmax(self.fc3(x), dim=1)

        return x

# Instantiate the model
model = SimpleNN()

# Print the model architecture
print(model)
```

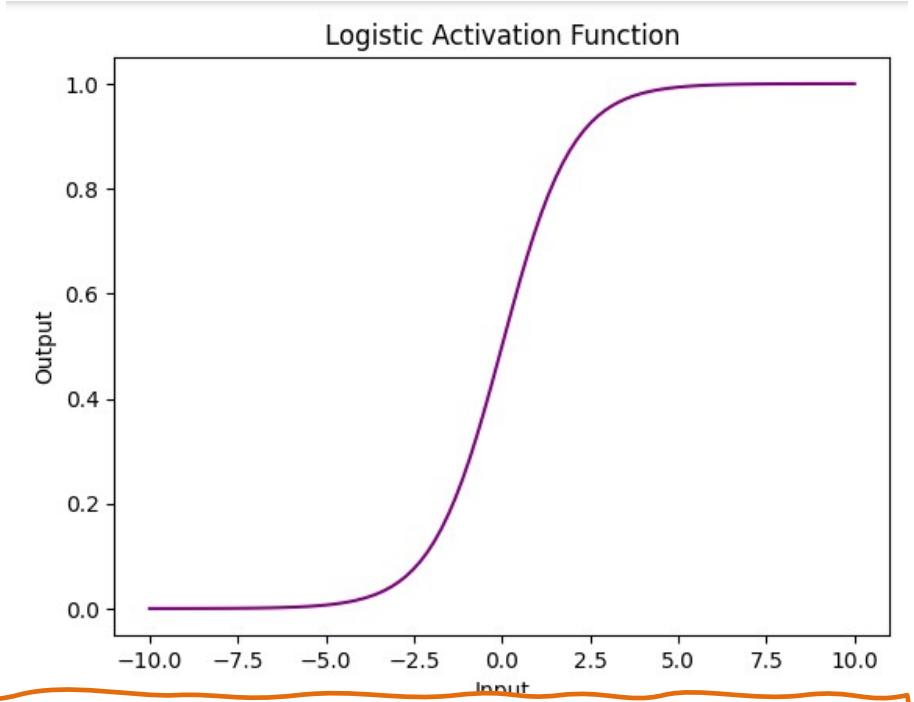
- **Logistic** (or **sigmoid**) is a non-linear activation function.
  - Useful for **binary classification tasks**. It takes any input and maps it to a value between 0 and 1, making it useful for **probability predictions**.
- **Tanh** is a non-linear activation function that outputs values between **-1** and **1**, with a mean output of **0**.
  - Ensure that the output of a neural network layer remains centered around 0, making it useful for **normalization**; can be **computationally expensive**
- **ReLU (Rectified Linear Unit)** is a non-linear activation function that outputs the input value if it is positive, or 0 if it is negative. computationally more efficient, **well-suited for large-scale neural networks**.

- **Linear** is a module provided by PyTorch that applies a linear transformation to the incoming data.
- **Non-linear activation** functions allow neural networks to capture complex patterns in data, which is essential for tasks like image recognition, natural language processing, and other AI applications.

Logistic (or **sigmoid**) example

!pip install matplotlib **on Jupyter Notebook will install it on Anaconda.**

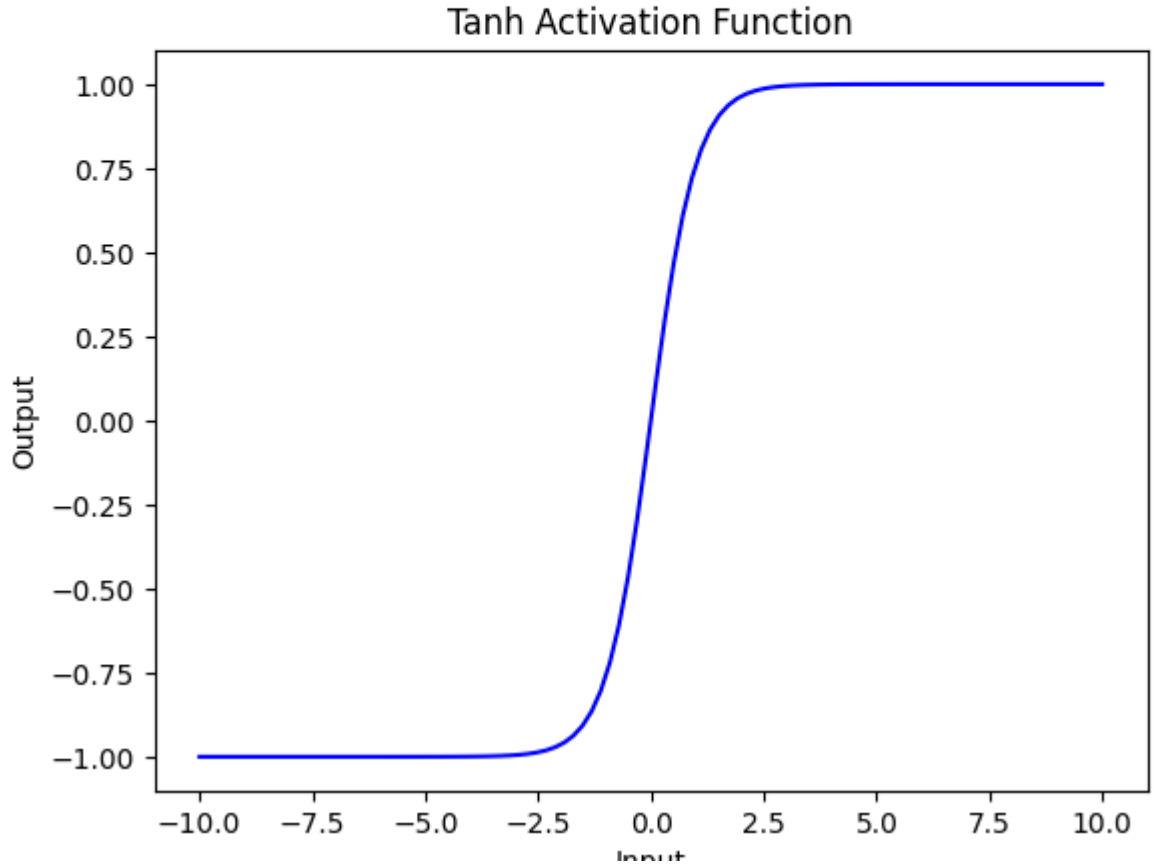
```
# importing the libraries
import torch
import matplotlib.pyplot as plt

# create a PyTorch tensor
x = torch.linspace(-10, 10, 100)

# apply the logistic activation function to the tensor
y = torch.sigmoid(x)

# plot the results with a custom color
plt.plot(x.numpy(), y.numpy(), color='purple')
plt.xlabel('Input')
plt.ylabel('Output')
plt.title('Logistic Activation Function')
plt.show()
```



-10: This is the start value of the tensor. The first element in the tensor x will be -10.
10: This is the end value of the tensor. The last element in the tensor x will be 10.
100: This specifies the number of equally spaced values between -10 and 10. The tensor x will contain 100 elements.

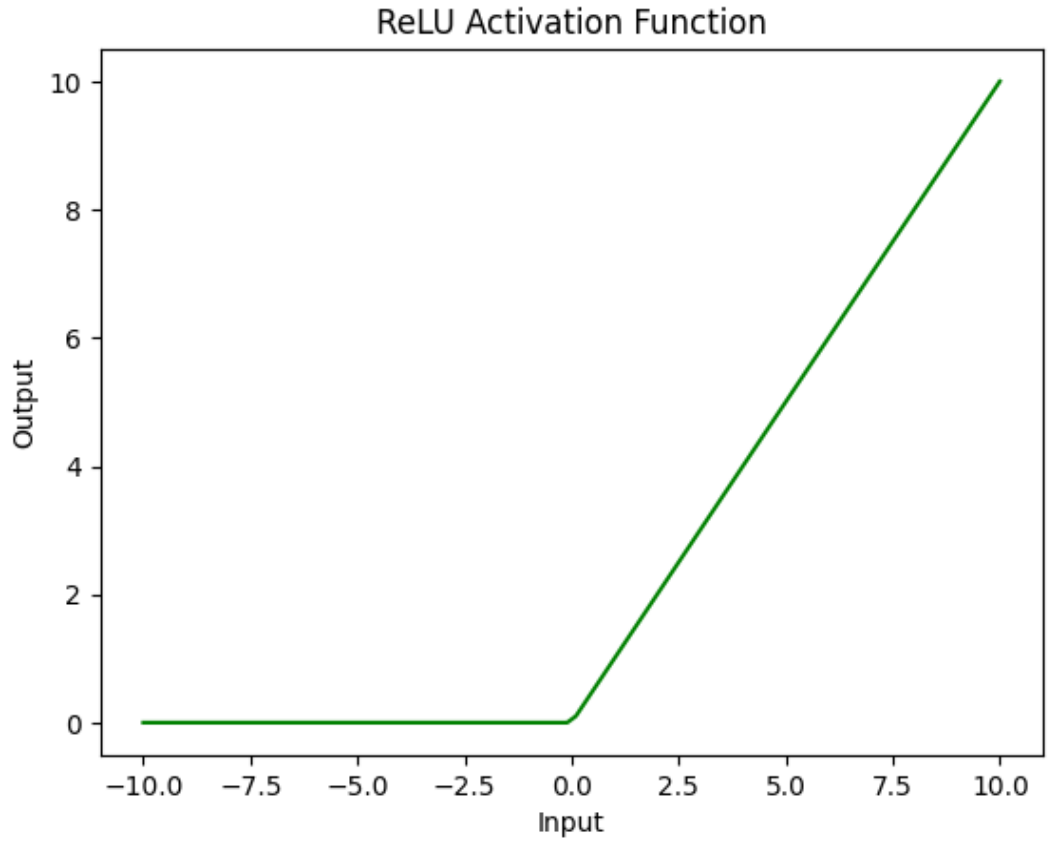
Tanh Activation Function example

```
# apply the tanh activation function to the tensor
y = torch.tanh(x)

# plot the results with a custom color
plt.plot(x.numpy(), y.numpy(), color='blue')
plt.xlabel('Input')
plt.ylabel('Output')
plt.title('Tanh Activation Function')
plt.show()
```



ReLU Activation Function

```
# apply the ReLU activation function to the tensor
y = torch.relu(x)

# plot the results with a custom color
plt.plot(x.numpy(), y.numpy(), color='green')
plt.xlabel('Input')
plt.ylabel('Output')
plt.title('ReLU Activation Function')
plt.show()
```



Phase 1 - Define the loss function measures how well the neural network's predictions match the actual target values. It quantifies the error in the predictions, which the training process aims to minimise.

- Common loss function: **Cross-Entropy Loss** used for classification tasks. **Mean Squared Error (MSE**) used for regression tasks.

Phase 2 - Select an optimizer. The optimizer updates the neural network's weights based on the gradients of the loss function with respect to the weights. The goal is to minimize the loss function by iteratively adjusting the weights.

- Common optimizers: Stochastic Gradient Descent; Adam (Adaptive Moment Estimation)).

Phase 3 - Implement the training loop: Core process where the neural network learns from data. It involves feeding input data through the network, calculating the loss, and adjusting the weights to minimize the loss.

The steps are:

- **forward pass**: Pass the input data through the network to obtain predictions. outputs = model(inputs)
- **Compute loss**: Calculate the loss using the loss function. loss = criterion(outputs, targets)
- **backpropagation**: Compute the gradients of the loss with respect to the model's parameters. loss.backward()
- **Update Weights**: Use the optimizer to update the weights based on the computed gradients. optimizer.step()

Summary

- Conda
- Google Colab
- PyTorch